AI ACADEMY · LEVEL 2 · GRADE 7 · WEEK 5 · STUDENT LESSON

Loops and Stopping

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Transparent Book Recommender

THE CONNECTED WEEK

Check the condition. Make progress. Explain the stop.

Move from fixed ranges to while-loops, trace boundary crossing and expose updates that cannot meet an equality-only stop.

Keep one habit: Test before updating → record progress → keep the final false check.

MAIN LESSON · SECTION 1

What makes this loop stop?

> Trace the condition, the update and the final state.

Start with progress toward a false condition

A while-loop checks its condition before each update. Explain how the update makes that condition false; reaching the boundary exactly is not always necessary or possible.

Recall changing state

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

Starting with total = 2, what does total = total + 3 store?

Your learning goals

- I can trace variable values before and after each **loop iteration**.
- I can explain how a while-loop update makes its stopping condition false.
- I can test zero, one and several while-loop updates and inspect a non-progress defect.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

Your learning target

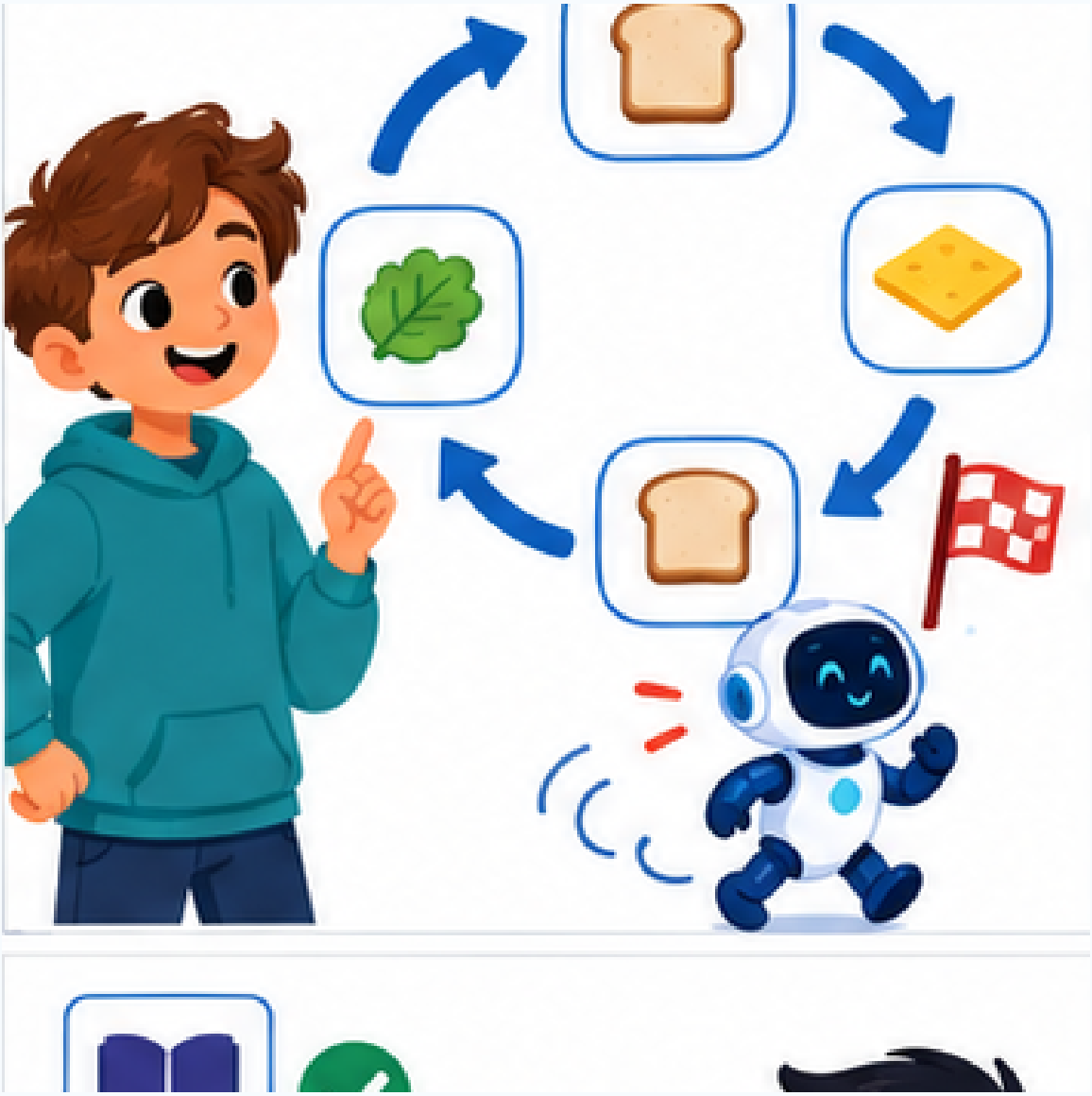
Remember before reading

Without opening the old lesson, explain one key idea from **Trace Table**s. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Decompose a Goal. Give an example and a mistake that the idea helps you avoid.

Week 5: **Loop**s and Stopping

Useful repetition needs a clear **update** and **stopping rule**.
</details>



THINKER MISSION

Predict **loop** counts; repair one **infinite loop**.

Thinker name and date:

> **Write or draw your reasoning and evidence.**

Readiness for the connected explanation

This week's end goal

By the end, I can trace a while-**loop** and recognise when its **update** cannot satisfy the stopping test.

MAIN LESSON · SECTION 2

Read the condition-controlled counting task

> Keep the earlier range examples as review, separate from the main while-loop.

The complete while requirement

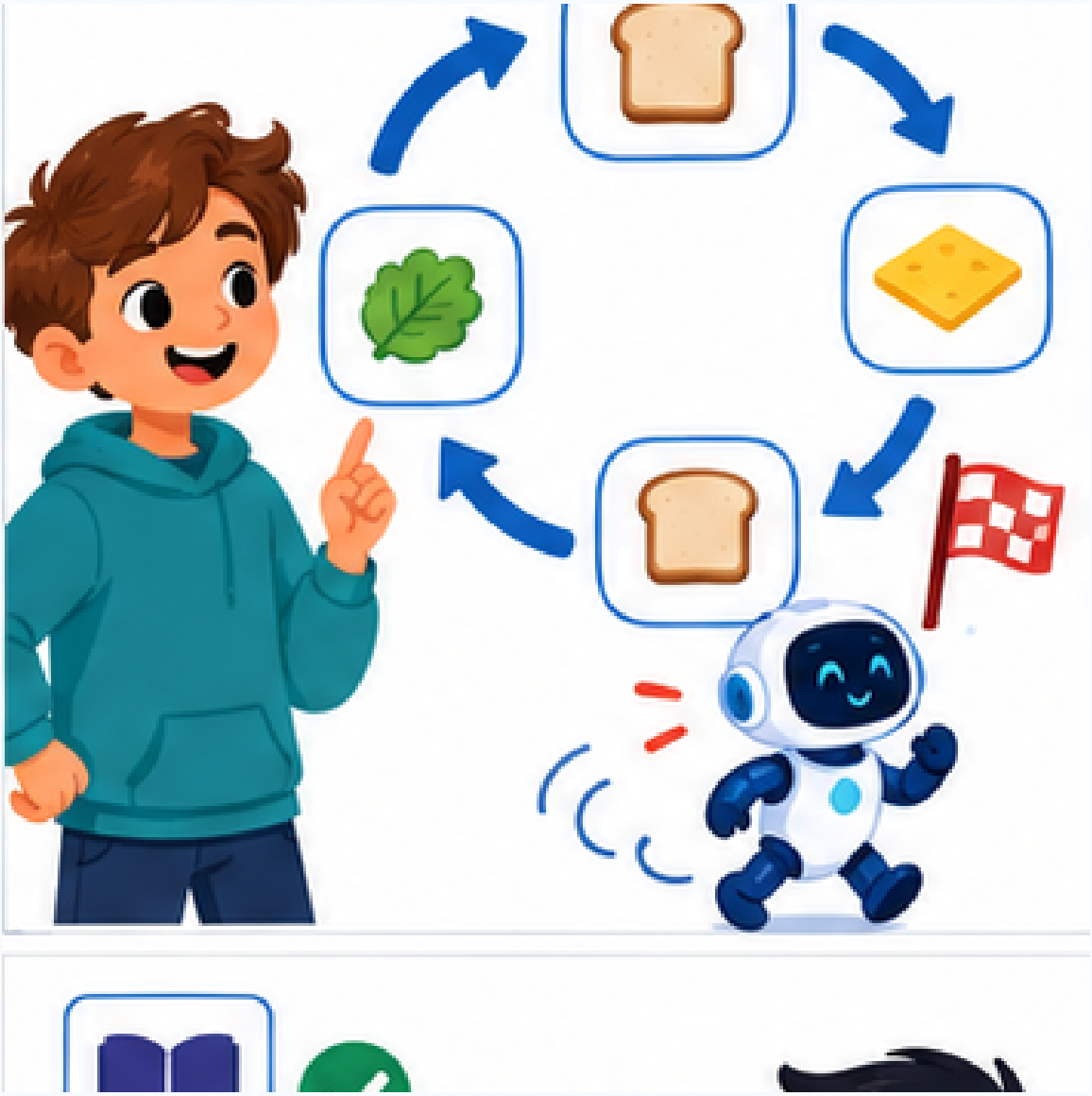
This week moves from the fixed ranges of Week 4 to a condition-controlled while-loop. Our supplied integer count starts at 12. While it is greater than 3, subtract 4. Check the condition before every subtraction; once it is false, print the current count once. The count may cross below 3 without ever equalling 3.

▼ **Earlier wording, alternatives and picture transcripts**

PICTURE STORY

Notice → Predict → Explain

Begin with visible **evidence**. Do not explain more than the picture and information support.



MYSTERY

The learners must use **loop**s and stopping to solve a problem. What information do they have? What decision or calculation do they make? What would count as **evidence** that it worked?

Three **observation**s (what you can point to):

> **Write or draw your reasoning and evidence.**

My **prediction** and the clue supporting it:

> **Write or draw your reasoning and evidence.**

A closer explanation

Imagine adding numbered counters: first 1, then 2, then 3. A **loop** can repeat the same addition with a different number each time. This week you will predict each change before running a short Python script.

Tracing repeated calculations prepares you to inspect the data **process**ing used in larger AI applications.

In for turn in range(1, 4), Python supplies 1, 2 and 3 in order. The start is included and the stop is excluded. The indented body runs once for each supplied value. The colon and four-space indentation mark that body.

This lesson uses integer range values with the default positive step of 1.

MAIN LESSON · SECTION 3

Trace each check before its update

> Keep the tested value, truth result and next state visible.

The complete executable while script

```
count = 12
while count > 3:
    count = count - 4
print(count)
```

Inspect all checks, including the final false one

Count before test	count > 3	Next action
12	True	Update to 8
8	True	Update to 4
4	True	Update to 0
0	False	Print 0 and stop

Explain indentation, progress and zero updates

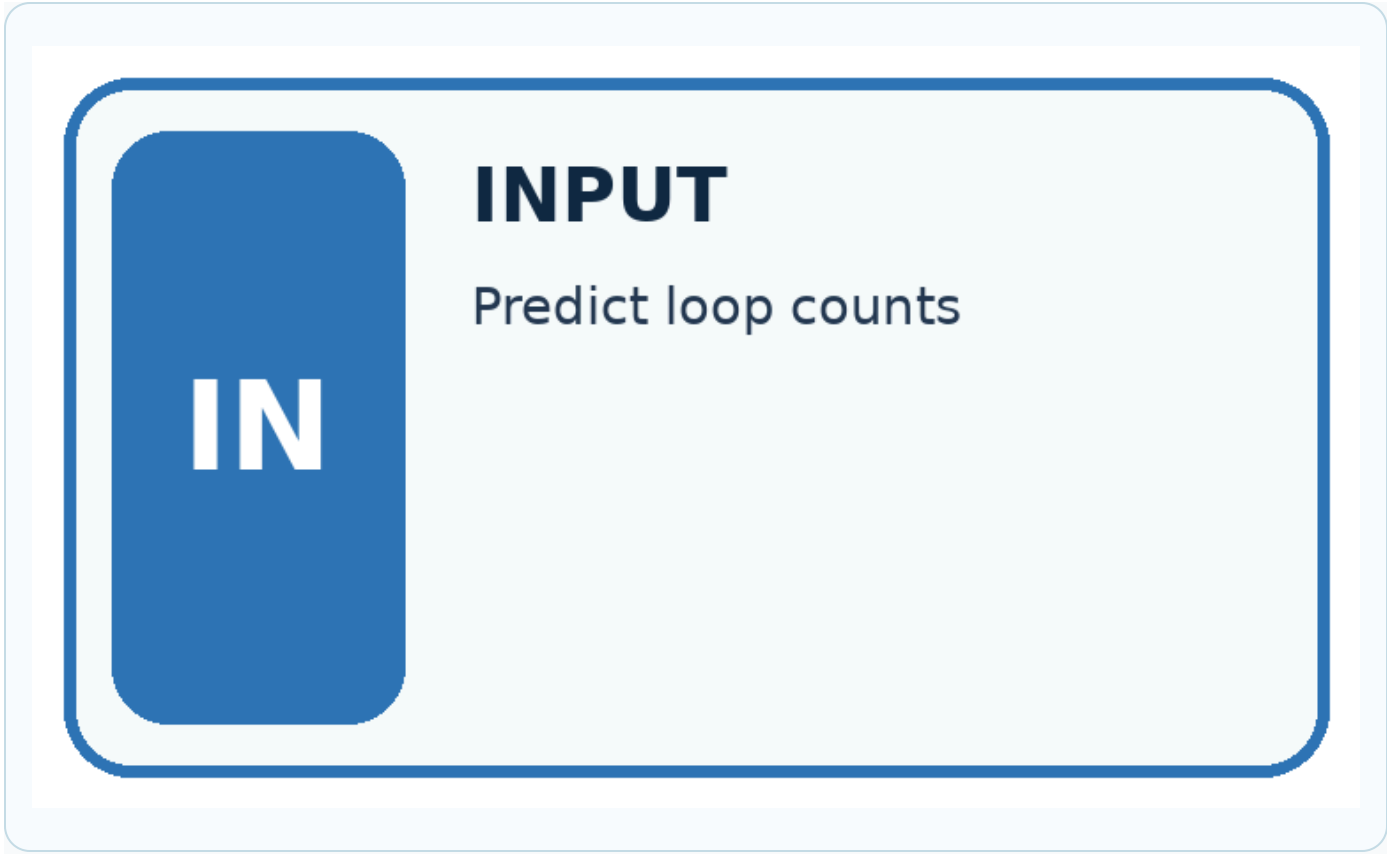
- The indented subtraction runs only after a true check. The final print is outside the loop. Subtracting four makes progress toward count≤3; it does not promise a final count of exactly three. With initial count 3, the condition is already false and there are zero updates.

▼ Earlier wording, alternatives and picture transcripts

VISUAL **MECHANISM**

Input → **Process** → **Output**

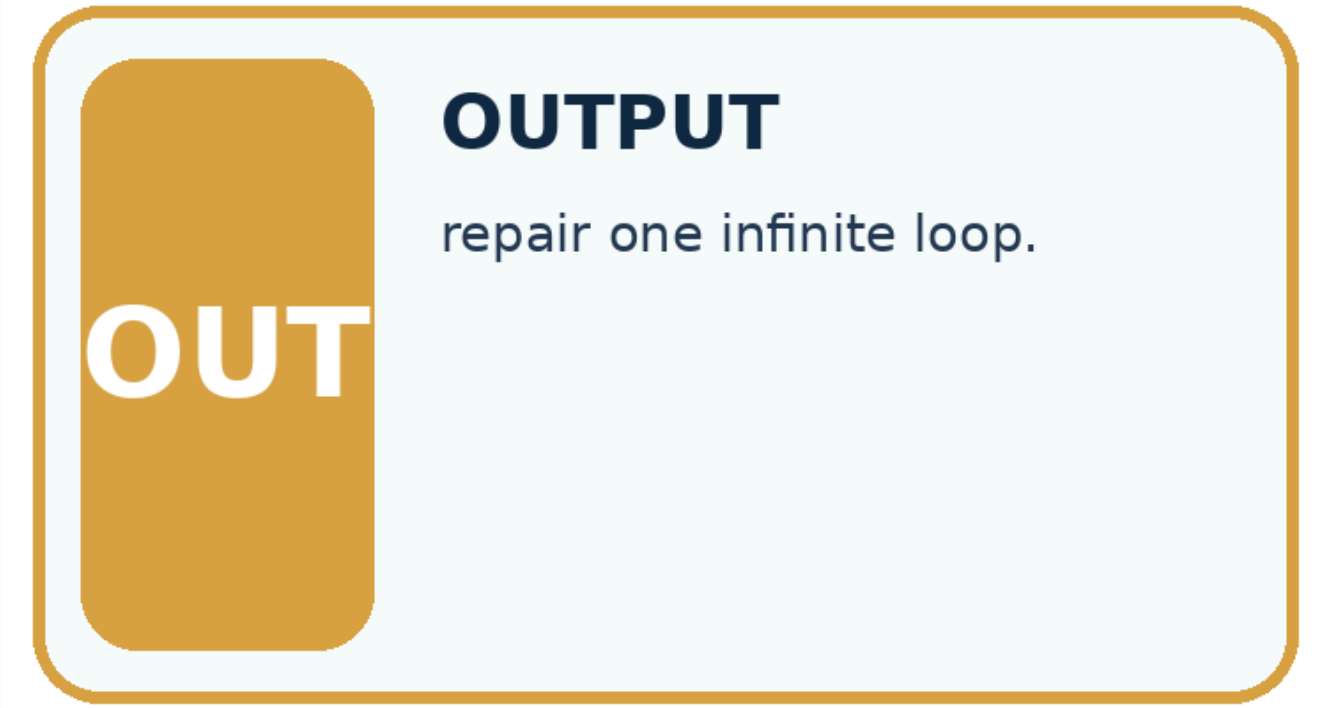
Trace the information. At each stage, ask what changed and how you could test it.


IN
INPUT
Predict loop counts

↓


PROCESS
Useful repetition needs a clear update and stopping rule.

↓



Which stage could fail even when the other two are correct? Explain:

> **Write or draw your reasoning and evidence.**

A closer explanation

```
total = 0
for turn in range(1, 4):
    total = total + turn
print(total)
```

State after each addition

TURN	TOTAL BEFORE	TOTAL AFTER
1	0	1
2	1	3
3	3	6

Initialize total to 0 before the loop. The unindented print runs once afterward.

The **output** is 6. Before the first **iteration**, total is 0. Each assignment reads the previous total and adds the current turn. After turn 3, the range is exhausted, so execution continues after the **loop**.

The table records state; it does not create extra instructions.

Connect the picture and the explanation

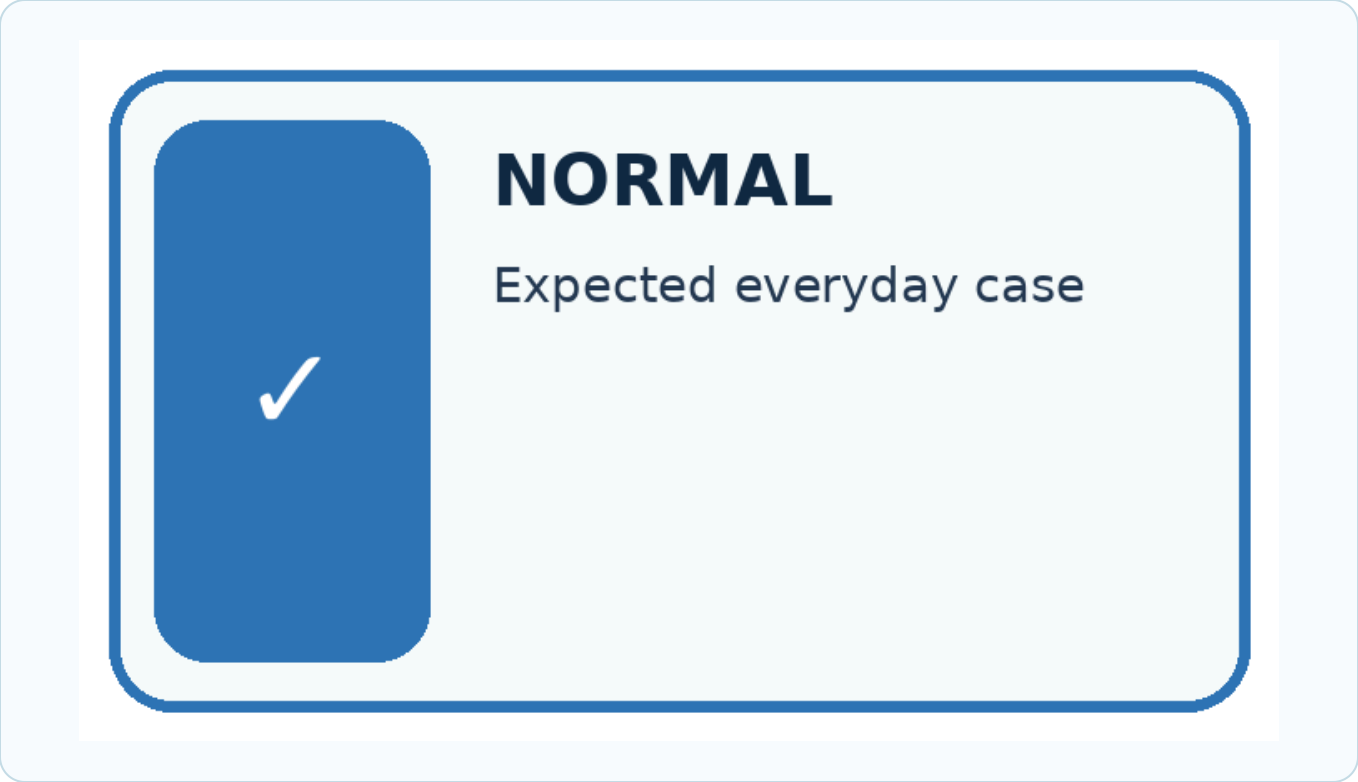
Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

MAIN LESSON · SECTION 4

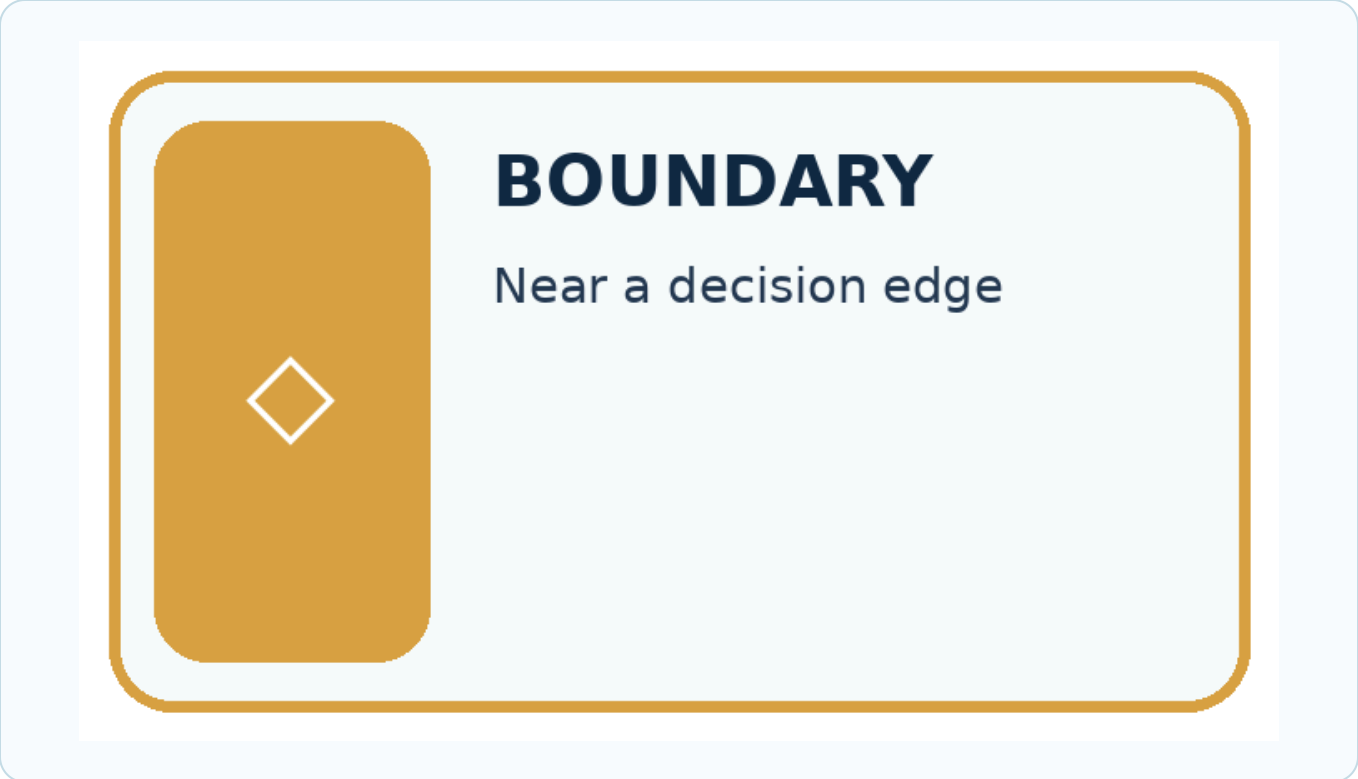
Compare ordinary, zero-update and non-progress cases

> An initially false condition can hide a missing update.

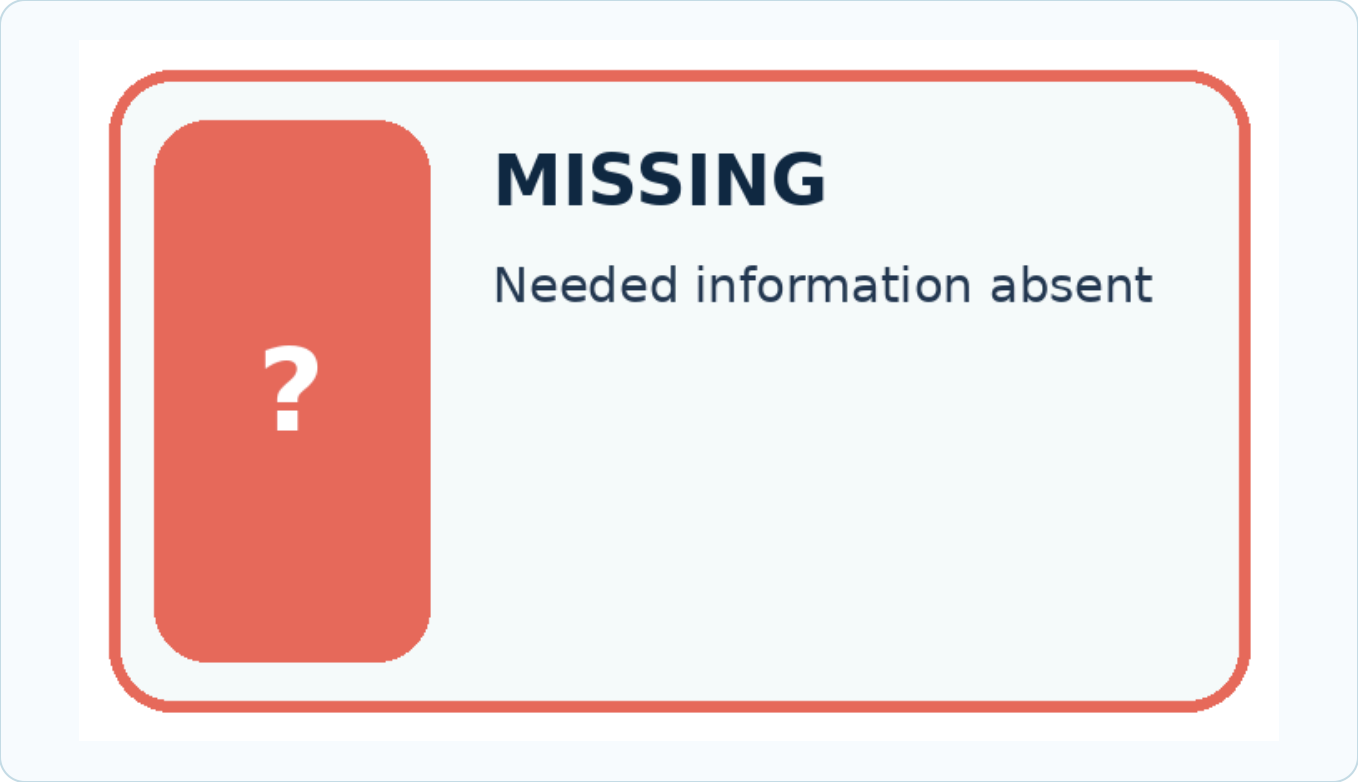
Four pictures with a purpose



Normal case: predict the result and deciding evidence.



Boundary case: identify the exact limit.



Missing information: name what must be clarified.



Transfer case: explain which procedure still applies.

Choose a revealing case

- Which case is most likely to reveal a hidden assumption? Why?

> **Write or draw your reasoning and evidence.**

Predict the supplied changed case

- Change: Replace count>3 with count!=3.

▼ Check after predicting

Prediction: From 12, the **loop** never reaches 3 and does not terminate in this toy model.

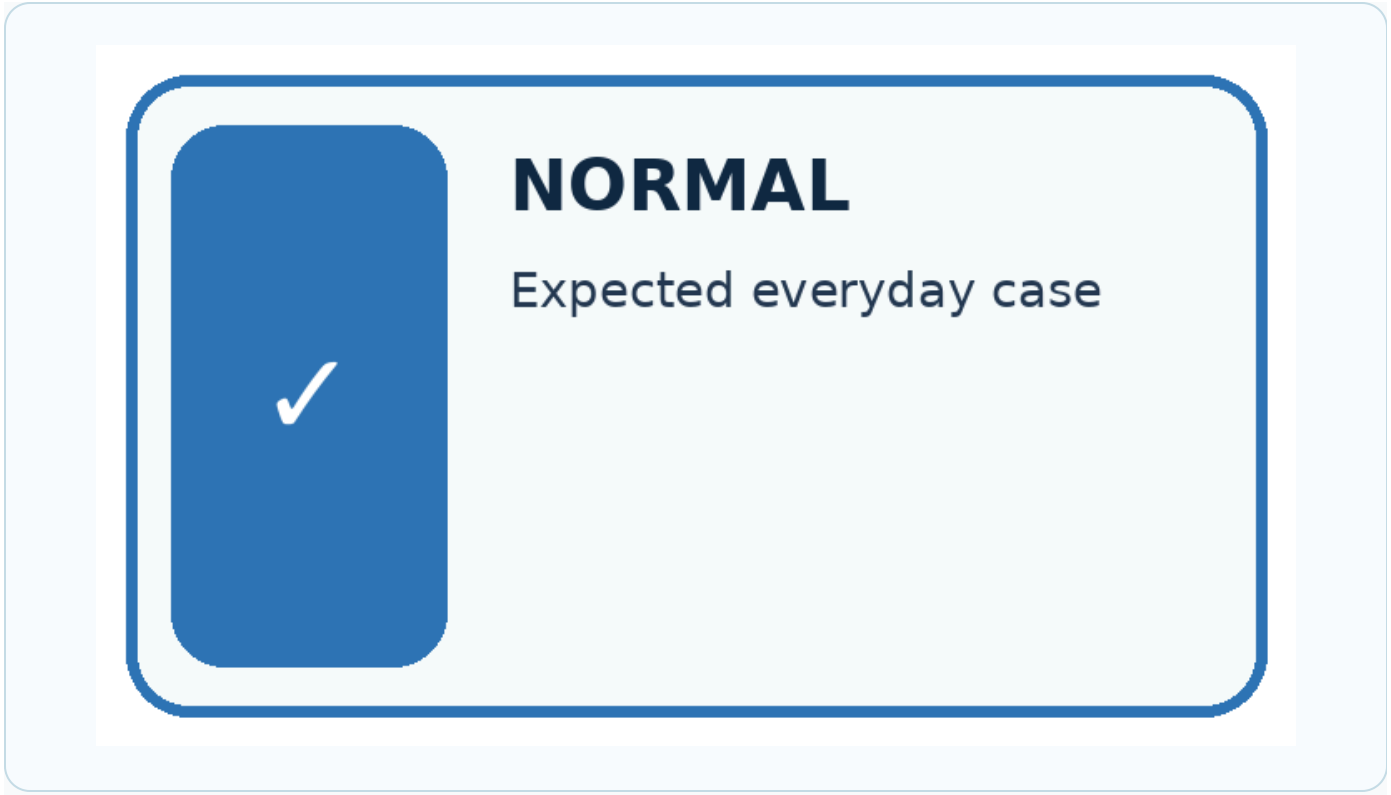
Reason: 12,8,4,0,−4,… are multiples of 4; none equals 3.

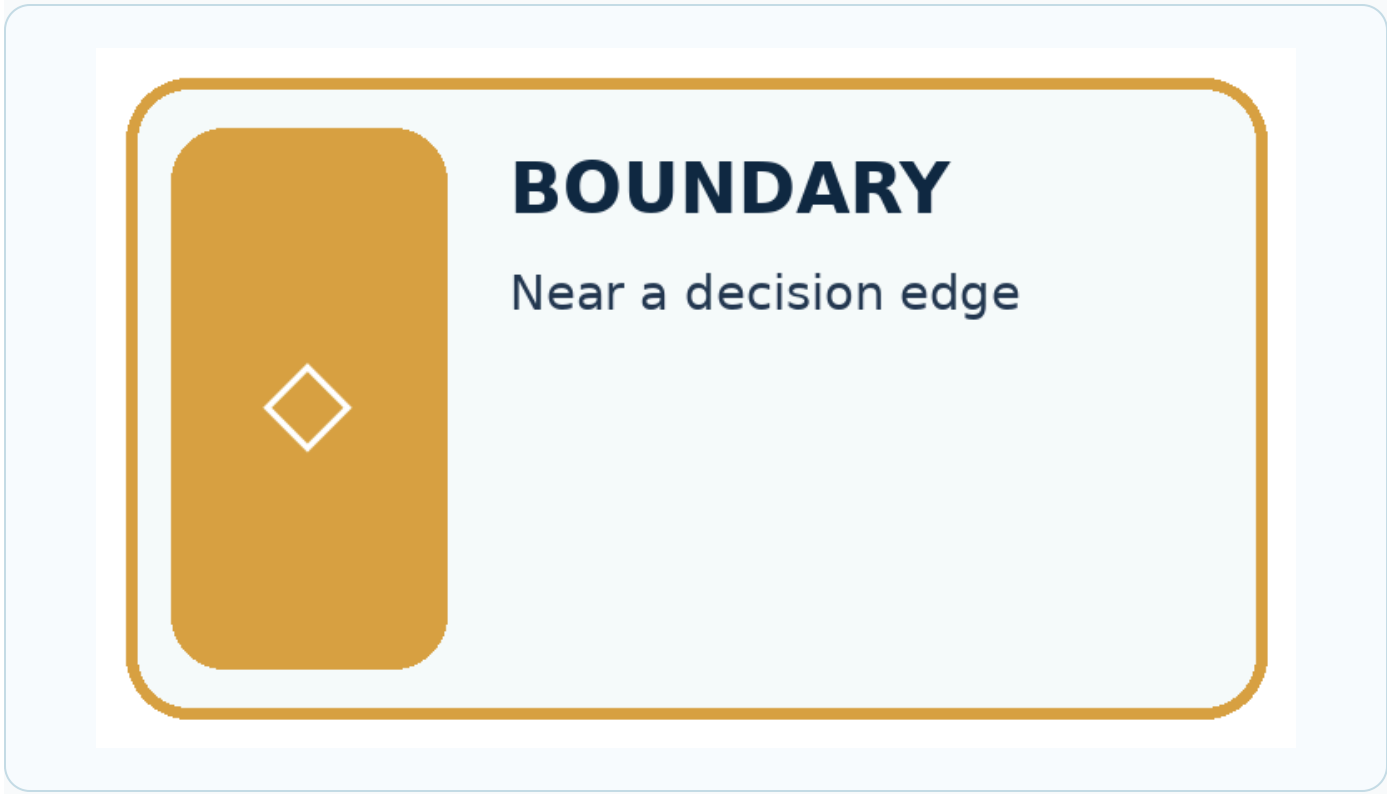
▼ Earlier wording, alternatives and picture transcripts

CASE GALLERY

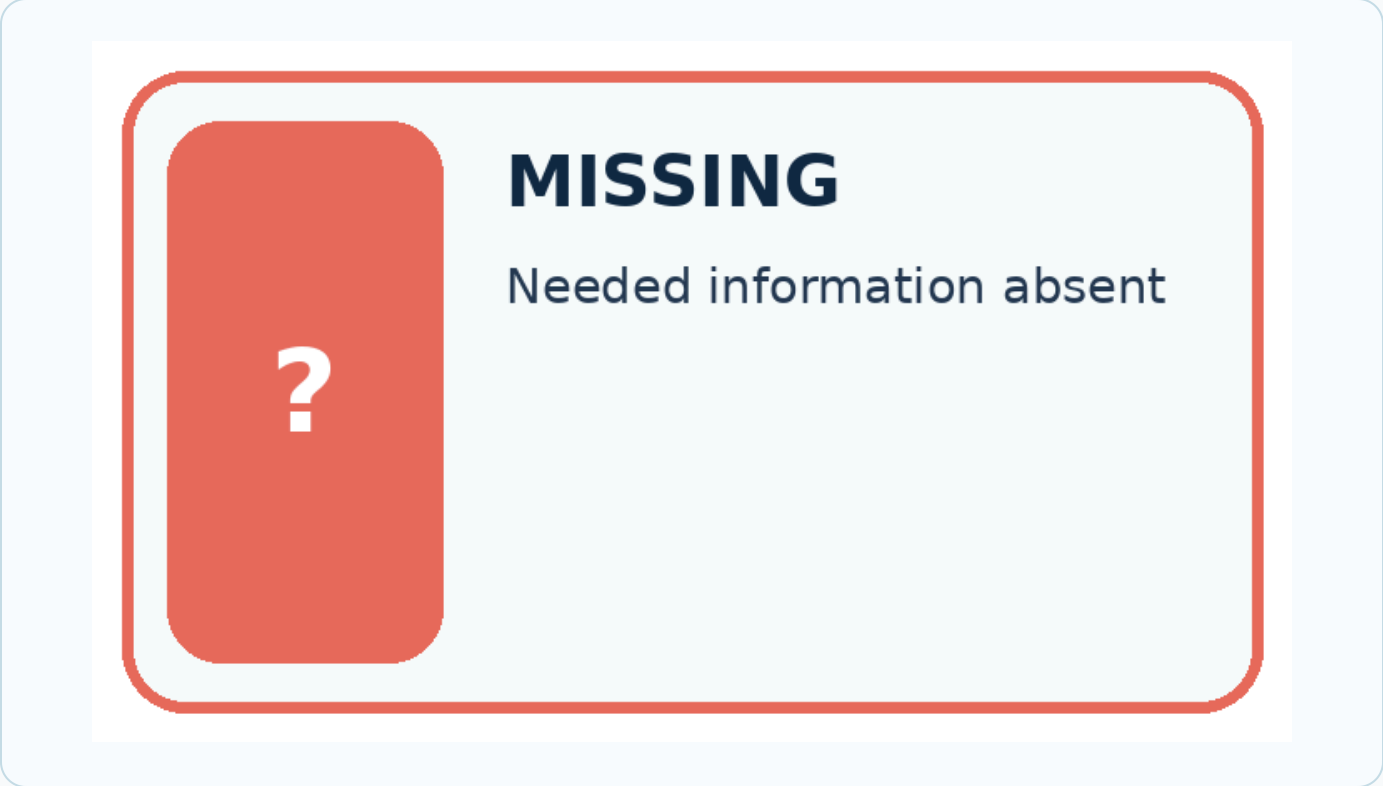
Four Tests, Four Kinds of **Evidence**

A system is not understood until it survives more than one comfortable example.





NORMAL CASE: predict and name **evidence**. BOUNDARY CASE: predict and name **evidence**.





MISSING INFORMATION: predict and name **evidence**. TRANSFER CASE: predict and name **evidence**.

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Explain crossing the boundary and skipping equality

> Use the same initial count and update as the main script.

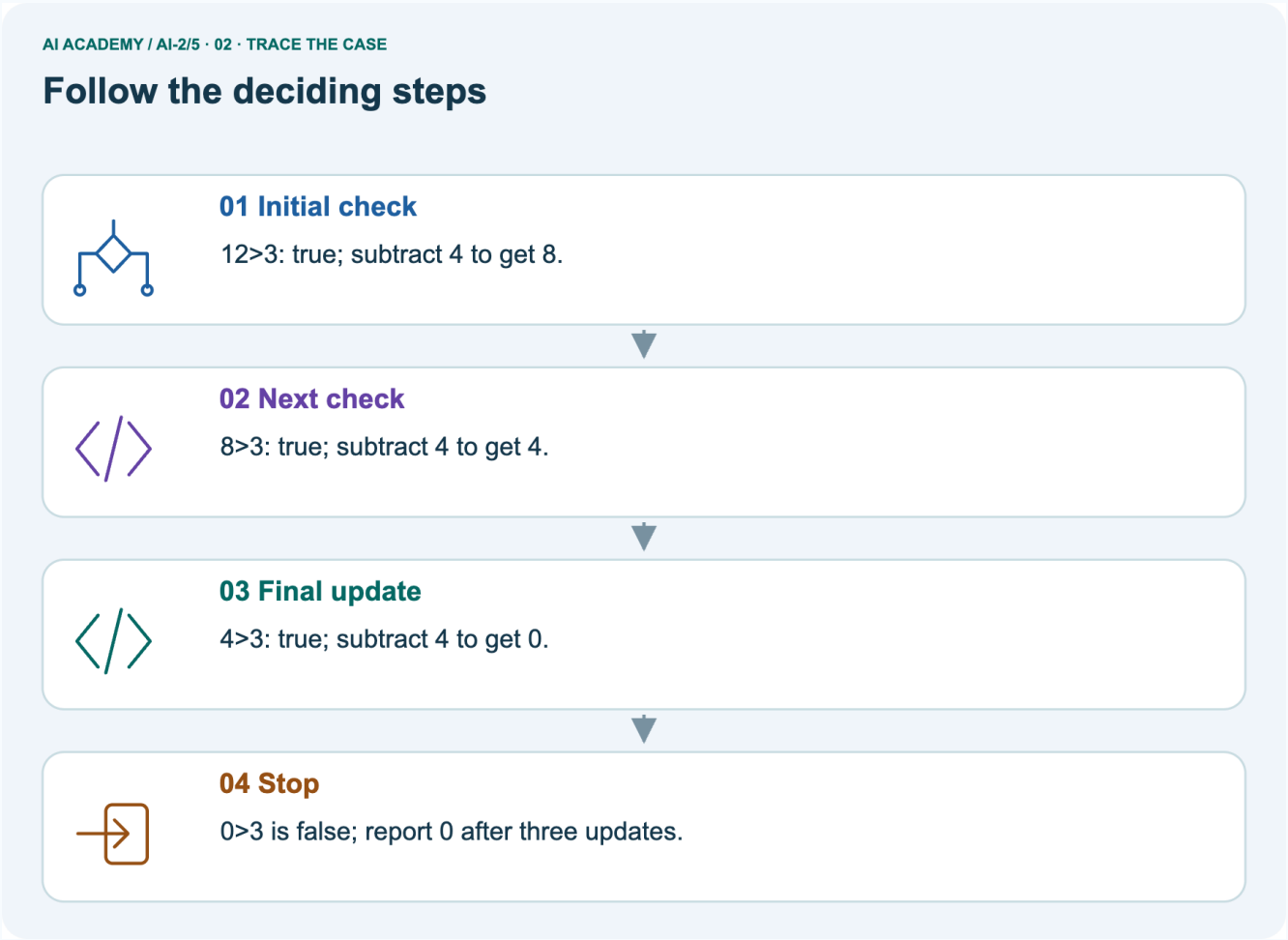
Read the counter facts

A fictional counter starts at 12 and decreases while above a boundary.

Python: count=12; while count>3: count=count-4; print after the **loop**.

The condition is checked before each subtraction.

Trace the three updates and final false check



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Initial check	12>3: true; subtract 4 to get 8.
2. Next check	8>3: true; subtract 4 to get 4.
3. Final update	4>3: true; subtract 4 to get 0.
4. Stop	0>3 is false; report 0 after three updates.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

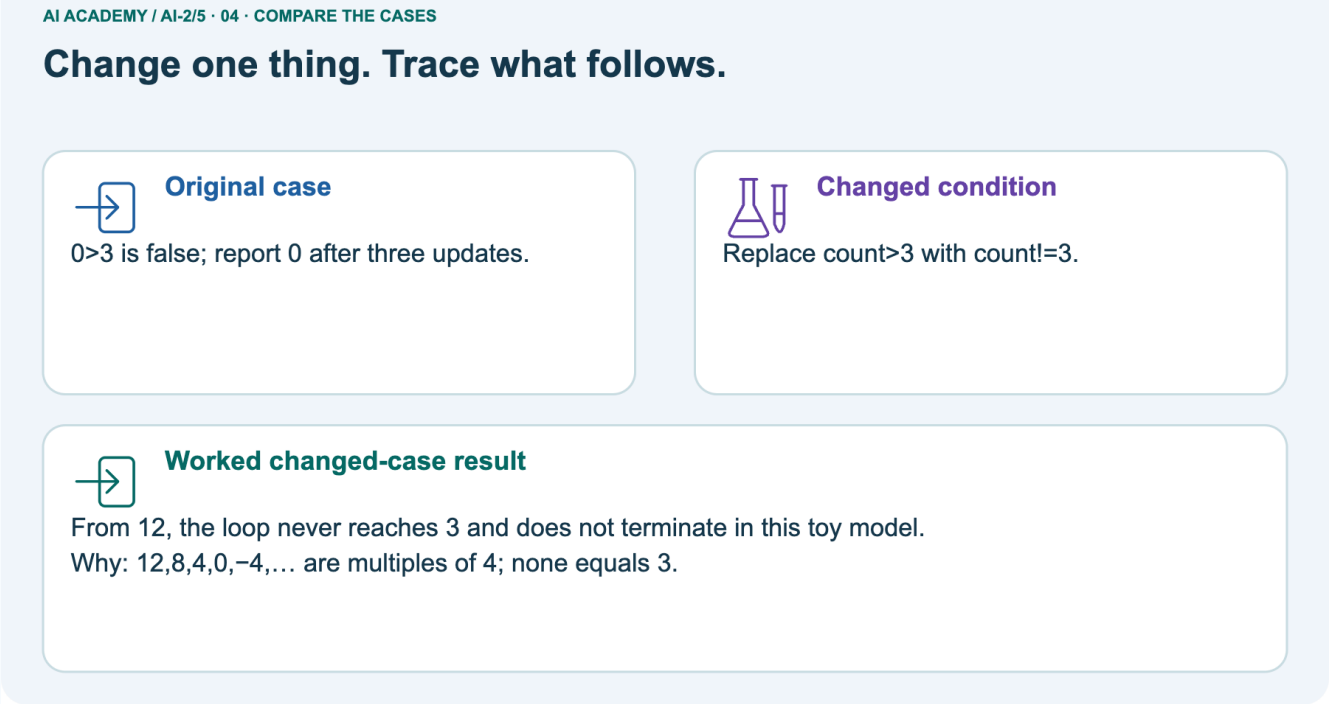
The condition does not promise the counter ends exactly at 3. Subtractions can cross the boundary. The count strictly decreases, so the below-or-equal stopping region is eventually reached from this start.

A condition requiring exact equality can be unsafe when repeated **update**s skip that value.

Predict equality-only stopping on paper

- Before opening the equality-only picture, replace >3 with !=3 while keeping the initial 12 and subtracting four. On paper, trace five updates only. Can any multiple of four equal three? Do not execute that unbounded version.

▼ Check why equality is never reached



Follow the labelled picture with the matching explanation. Open the vector diagram

Explain the final zero and finite-prefix limit

- Read the labelled picture: Why is report 0 correct for the original **loop**, and why is equality-only stopping risky here?
- Do not run an intentionally unbounded **loop** in class; trace a finite prefix to inspect the defect safely.

▼ Earlier wording, alternatives and picture transcripts

WORKED **EVIDENCE**

Reason Step by Step

Predict first. Then check each step. A correct answer without visible reasoning is incomplete **evidence**.

WEEK 5 • WORKED EVIDENCE

1. Start count = 0.
2. Repeat while count < 4: show a card; count = count + 1.
3. The loop runs 4 times and stops when count becomes 4.
4. If count never changes, the stopping condition never becomes false.

1. Start count = 0.

2. Repeat while count < 4: show a card; count = count + 1.

3. The **loop** runs 4 times and stops when count becomes 4.

4. If count never changes, the stopping condition never becomes false.

Recalculate, retrace, or restate the reasoning in your own representation:

> **Write or draw your reasoning and evidence.**

CHECK Did the **evidence** answer the exact question? Did the total, path, or source record stay consistent?

A closer explanation

Range expression	Values supplied	Number of iterations
range(1, 4)	1, 2, 3	3

Range expression	Values supplied	Number of iterations
range(1, 2)	1	1
range(1, 1)	No values	0
range(4)	0, 1, 2, 3	4

With one argument, range starts at zero. range(4) supplies four values but none is 4. If the worked script uses range(1, 1), the body never runs and the initialized total remains 0.

Do not rely on a variable first assigned inside a **loop**: a zero-**iteration** run might never assign it.

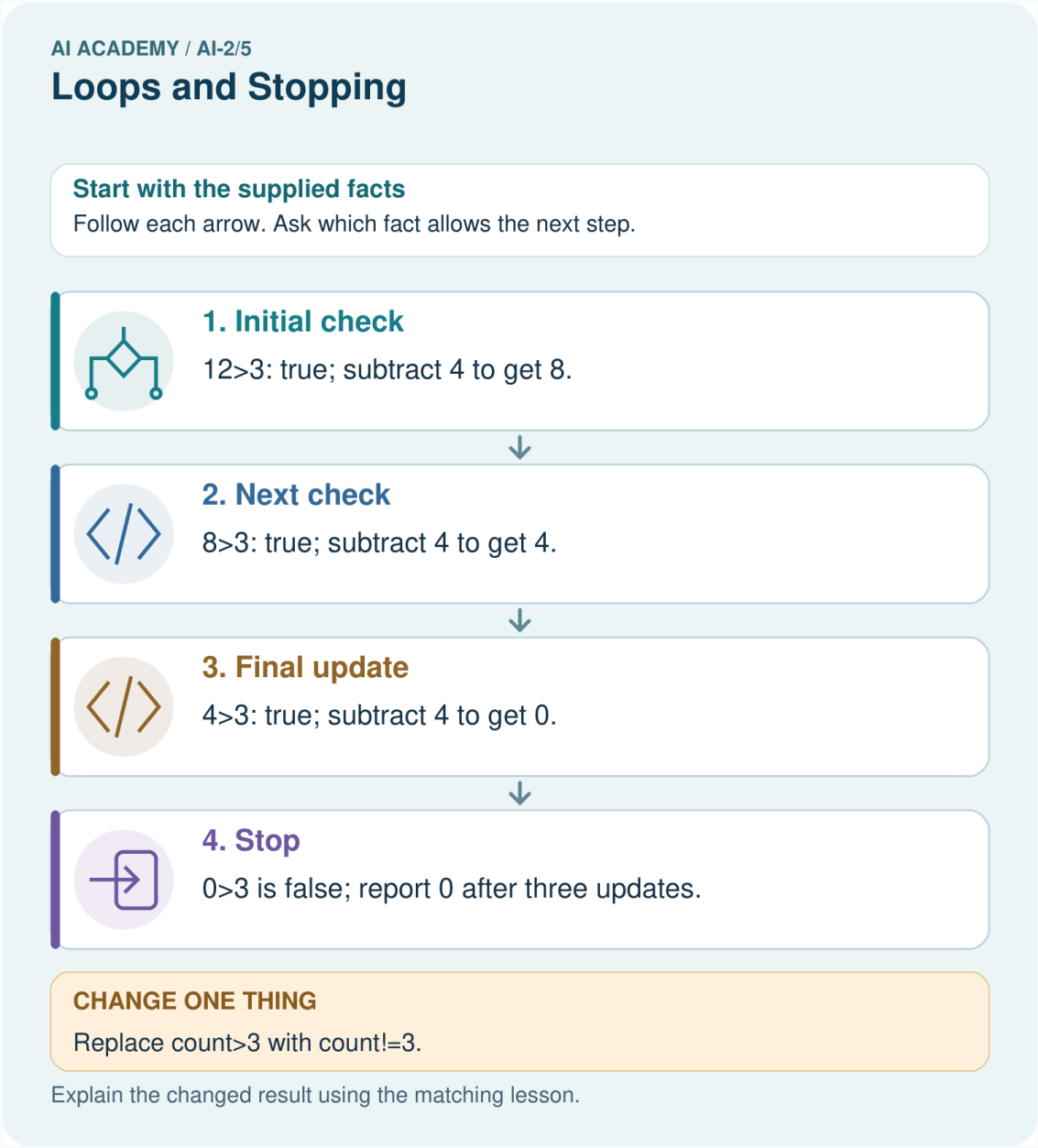
> **Replace range(1, 4) with range(1, 3) in the worked script. Predict the supplied values and final total. What stop value is needed to include 3?**

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Loops and Stopping

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

Change one thing: Replace count>3 with count!=3. Predict the result before checking the explanation. What remains the same?

MAIN LESSON · SECTION 7

Find the missing progress step

> A successful zero-update case is not evidence that the body makes progress.

Compare an initially false and initially true test

A faulty loop leaves count unchanged while testing count>3. With count=3 it stops immediately, hiding the defect. With count=12 the same true condition is tested forever in the toy model. Locate the missing update rather than changing the expected answer. Inspect a finite paper prefix instead of running an intentionally unbounded program.

Keep a first-cause repair record

- Repair one cause. Why is this repair better than changing several things?

> **Write or draw your reasoning and evidence.**

▼ Earlier wording, alternatives and picture transcripts

ERROR DETECTIVE

Compare → Diagnose → Repair

Do not patch the answer. Find the earliest unsupported assumption, wrong step, or weak **evidence**.

!

COMPARE • DIAGNOSE

Weak idea: Every repeated action will eventually stop by itself. Strong move: locate evidence, change one cause, and retest.

Evidence record	Expected	Actual	Likely cause / next test
normal case	works		
boundary case	stated rule		
missing input	safe response		
fresh transfer	generalizes		

A closer explanation

Write the expected range values and **trace table** before running. Keep predicted and observed **output** separate, then explain the first row where they differ.

For a **loop** that should add a fixed amount repeatedly, zero **iteration**s should preserve the initial amount. One **iteration** checks one **update**; several check accumulation.

> **A learner checks the reset bug using only range(1, 2) and sees the expected answer 1. Explain why this does not establish correctness, and give a revealing test with its expected correct output.**

MAIN LESSON · SECTION 8

Practise zero, one and several while updates

> Check the final false condition as well as the body.

The supported while test plan

1. Predict fresh runs of the correct main script with initial count 3,4,12. Record each tested count, condition result, updated count and final output. Then inspect the no-update and !=3 variants using finite paper traces. Restore only the faulty step and rerun the terminating cases; keep old and new records separately labelled.

Return to the boundary-crossing explanation

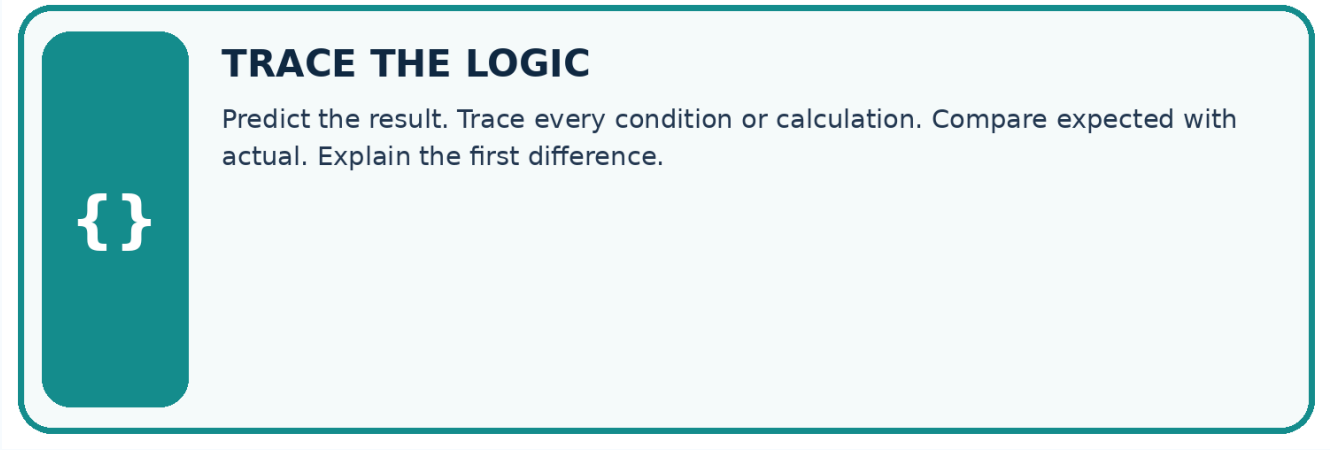
- Why is report 0 correct for the original **loop**, and why is equality-only stopping risky here?
- This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ Earlier wording, alternatives and picture transcripts

LOGIC STUDIO

Trace Before You Run

Block code, pseudocode, tables, and diagrams are different ways to expose the same reasoning.



SET **evidence**_record PREDICT expected_result RUN one normal case and one boundary case COMPARE actual_result with expected_result IF different: locate first cause; change one factor; RETEST EXPLAIN result, limitation, and next safe step

Trace one **input**. Which line changes the state or chooses a path?

> **Write or draw your reasoning and evidence.**

A closer explanation

```
total = 0
for turn in range(1, 4):
    total = 0
    total = total + turn
print(total)
```

This faulty version prints 3. The extra reset discards earlier additions on every **iteration**. Remove the indented reset while keeping **initialization** before the **loop**.

Then test zero, one and several **iteration**s: the one-**iteration** case alone cannot reveal this reset bug.

Moving print(total) into the correct **loop** produces three lines, 1 then 3 then 6. Moving it out produces only the final 6. Trace both values and **output** events.

A **loop** follows written instructions; repeating them does not make the program learn.

Practise with less help: Trace before running

Use workbook W2 for the connected example "**Loop**s and **trace table**s". First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Use the worked case for **Loop**s and **trace table**s. Proposed explanation: "The stop value is included". Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

> **My explanation using a pictured step and the given facts**

Investigate the update that makes stopping possible

> Use a bounded paper inspection for faulty non-progress cases.

The supplied paper and code stations

1. Use numbered paper cards to simulate count=0; while count<4, display a card and add one. Record tested counts before updates and displayed cards separately. Stop after the first false check. Remove the update only for a finite three-step paper inspection, then restore it. Next run the complete decreasing-count Python script with starts 3,4,12 and retain expected versus actual results. Use paper prefixes for intentionally nonterminating variants; never fill observations from worked predictions.

Keep predictions, actual results and reasons

Test / input · Prediction · Actual · Evidence / calculation · Change or keep?

> **Test / input:** normal
>
> **Prediction:** _____
>
> **Actual:** _____
>
> **Evidence / calculation:** _____
>
> **Change or keep?:** _____

> **Test / input:** boundary
>
> **Prediction:** _____
>
> **Actual:** _____
>
> **Evidence / calculation:** _____
>
> **Change or keep?:** _____

> **Test / input:** missing
>
> **Prediction:** _____
>
> **Actual:** _____

Evidence / calculation: _____

Change or keep?: _____

Test / input: fresh transfer

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

Explain what the revealing test established

- What did the hardest test teach you that the normal test did not?

- Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

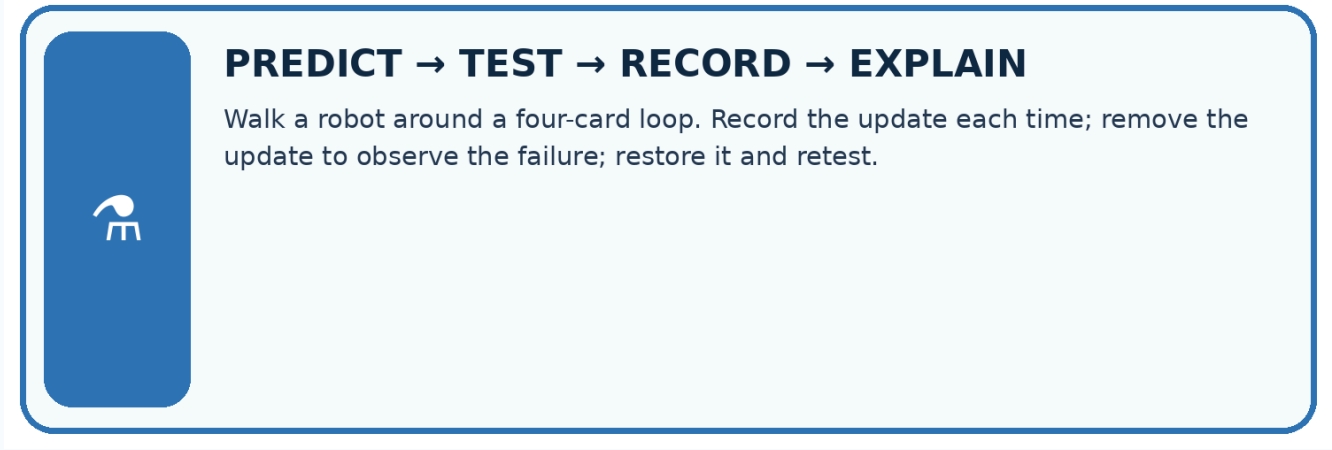
Write or draw your reasoning and evidence.

▼ Earlier wording, alternatives and picture transcripts

EVIDENCE LAB

Predict → Test → Record → Explain

Keep expected and actual results separate. Surprises are useful data when recorded honestly.



Walk a robot around a four-card **loop**. Record the **update** each time; remove the **update** to observe the failure; restore it and retest.

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: trace the different increasing loop

> Use its start, step and boundary before feedback.

The independent condition-and-update task

Start count at 1. While count < 8, add 3. Trace the condition and values. What happens if the condition becomes count != 8?

Keep your first attempt

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Your teacher has the independent answer and scoring criteria. Complete the matching workbook week to keep your practice **evidence** together.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ **Earlier wording, alternatives and picture transcripts**

Independent case: Changed independent case

Week 5 Independent application

Close the worked explanation. Apply the idea to this different question without copying.

Start at count 0. While count is less than 3, add 1 and display count. List the displays and the stopping value.

My answer, with a trace, calculation or supporting **evidence**

Write or draw your reasoning and evidence.

Create and test

Change one relevant detail. Predict the result before testing. Include a boundary or missing-information case when it helps reveal a limit.

My changed **input** and expected result

> **Write or draw your reasoning and evidence.**

Connect to your project

Loop through every candidate book, **update** its score, then stop at the end of the list.

The **evidence** I will keep

> **Write or draw your reasoning and evidence.**

Transfer practice from the original programme

Trace count=0; while count<4: count=count+2. List values tested by the condition and final count. Would replacing <4 with !=5 terminate?

> **Explain the mechanism, evidence and one limit.**

Use feedback to improve

MAIN LESSON · SECTION 11

Add a progress-and-stopping argument to the portfolio

> Use explicit candidate state rather than an unexplained repeat arrow.

Scope of this week's contribution

For the Transparent Book Recommender, repeated processing needs a current item, progress to the next item and an explicit end. This week contributes a stopping argument using fictional records and declared rules; it does not supply a scoring formula or train a model. Keep zero, one and several-item tests and human choice visible.

The checkpoint and system map

THIS WEEK'S CHECKPOINT **Loop** through every candidate book, **update** its score, then stop at the end of the list.

SYSTEM MAP Reader need → allowed **input**s → transparent rule or score → ranked candidates → explanation → test record → **human decision**

Use progress and stopping arguments for any repeated recommender **process**; test **input**s that require zero, one and several **update**s.

Record progress and human control

- What will you add, change, or verify in the project this week?
- What should remain under **human control**?

Write or draw your reasoning and evidence.

Write or draw your reasoning and evidence.

Keep privacy and ranking limits

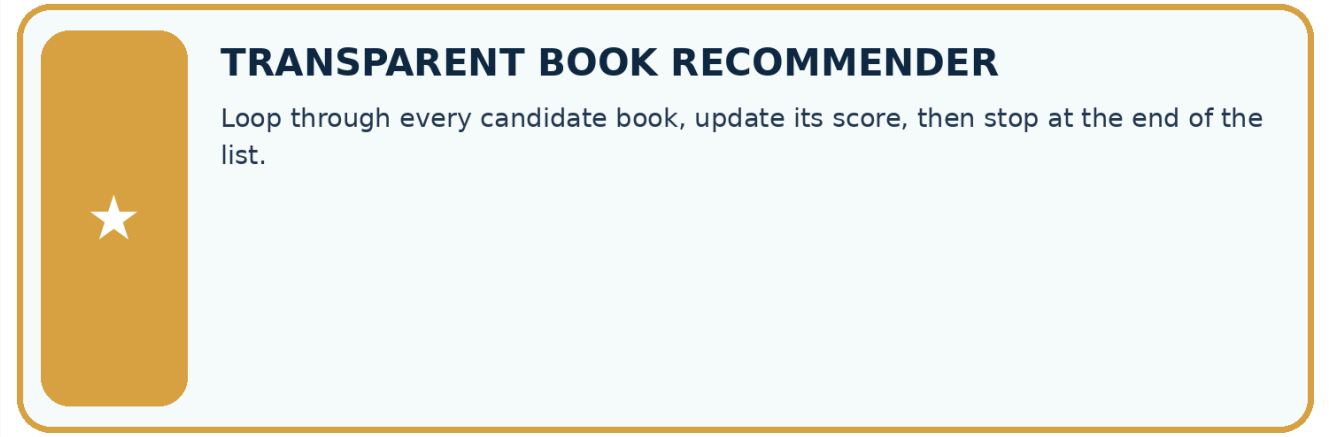
RESPONSIBLE DESIGN Collect only necessary information. Show uncertainty and limits.
Never confuse a ranking with a fact about a person.

▼ Earlier wording, alternatives and picture transcripts

CUMULATIVE PROJECT

Transparent Book Recommender

Every week adds one inspectable piece. The system must show how it reached a result.



Evidence for your project

Finish: justify termination from the condition and update

> Close your notes and distinguish boundary crossing from exact equality.

Your exit explanation

- Close your notes. Explain this goal in your own words: Trace variable values before and after each **loop iteration**

> **My deciding step, evidence and one remaining question**
>
> | |
> |---|
> | |
> | |
> | |
> | |

Choose a revealing start or update

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Your quick word guide**

REFERENCE · SECTION 5

Your quick word guide

> Use the words to explain a particular step, record or limit.

Core vocabulary

Word	My own example	Not this / boundary
loop	A structure that repeats instructions according to a count or condition.	
update	Change a stored value or state according to a specified operation or new evidence.	
stopping rule	A specified condition that ends a repeated process.	
infinite loop	A loop that never reaches its stopping condition under the assumed execution model.	

Terms for the connected explanation

Word	Meaning
Iteration	One execution of a loop body
Accumulator	A variable that keeps a running result

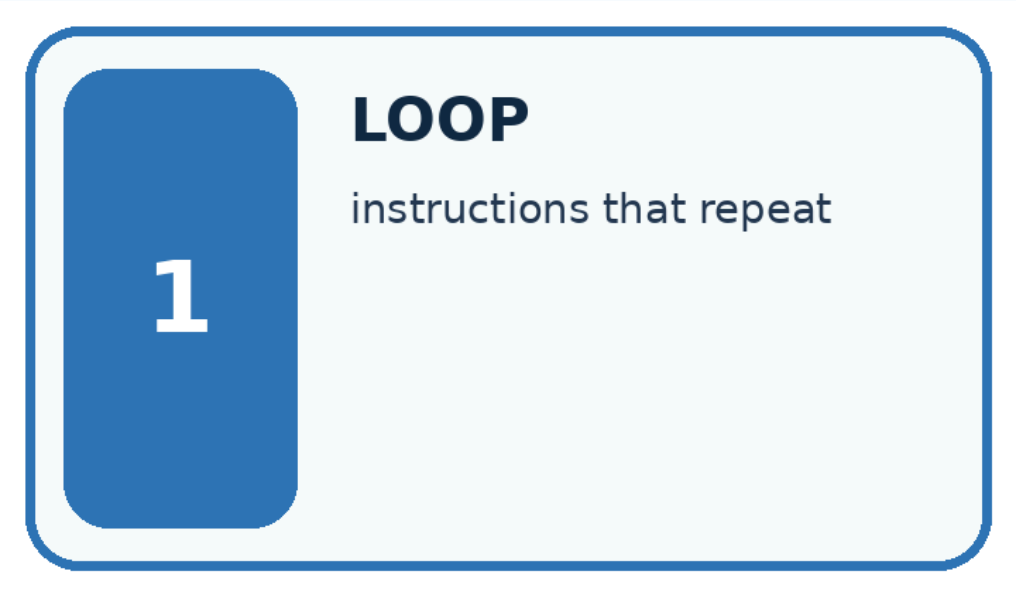
Word	Meaning
Initialization	Giving a variable its starting value
Trace table	A record of variable values as instructions execute

▼ Earlier wording, alternatives and picture transcripts

VOCABULARY LAB

Words You Can Use to Reason

A definition matters when you can recognize the idea, use it, and contrast it with a near-miss.



2

UPDATE

a change made each repetition

3

STOPPING RULE

condition that ends repetition



Words used in the closer explanation

▼ **Choose a stronger thinking challenge**

REFERENCE · SECTION 10

Choose a stronger thinking challenge

> Choose the next useful challenge rather than treating the whole ladder as compulsory.

Pick your next step

- 1 • NAME: Define the concept without repeating the textbook.
- 2 • TRACE: Show the information, state, branch, calculation, or **evidence** path.
- 3 • APPLY: Solve a different example independently.
- 4 • CHALLENGE: Find a boundary case, counterexample, or unfair outcome.

- 5 • EXPLAIN LIMITS: Say when the result should not be trusted or used alone.

- My strongest **evidence** of independent understanding:

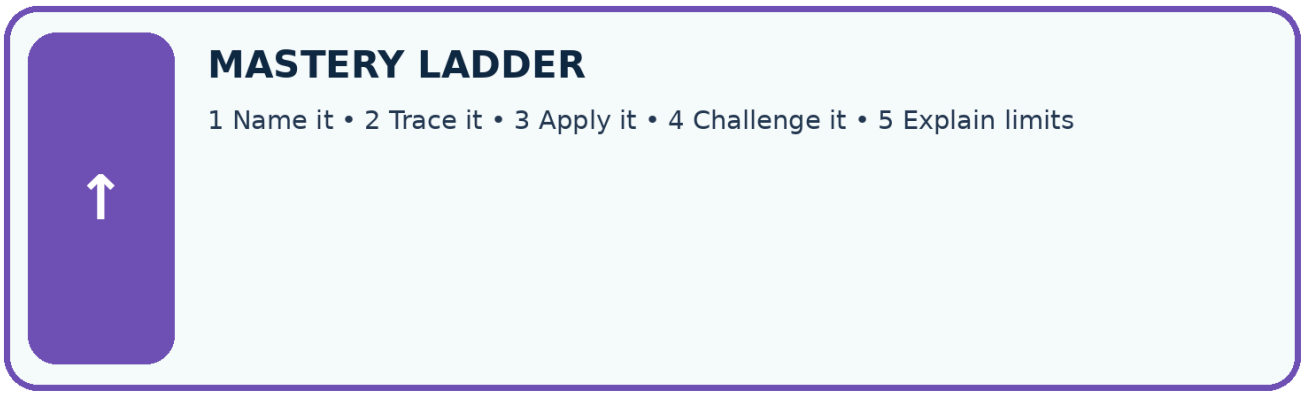
Write or draw your reasoning and evidence.

▼ Earlier wording, alternatives and picture transcripts

MASTERY

From Knowing the Word to Owning the Idea

Move upward only when you can produce **evidence** without copying the worked example.



▼ Reference: check your explanation

Reference: check your explanation

> Use after practice. Changed independent case is the selected independent case.

Check the evidence and its limit

- 1. My fresh example, trace, calculation, or design:
- 2. My **evidence** that it works (include a boundary or missing-data case):
- 3. One limitation, fairness concern, or situation needing human review:

> **Write or draw your reasoning and evidence.**

> **Write or draw your reasoning and evidence.**

Self-check and record

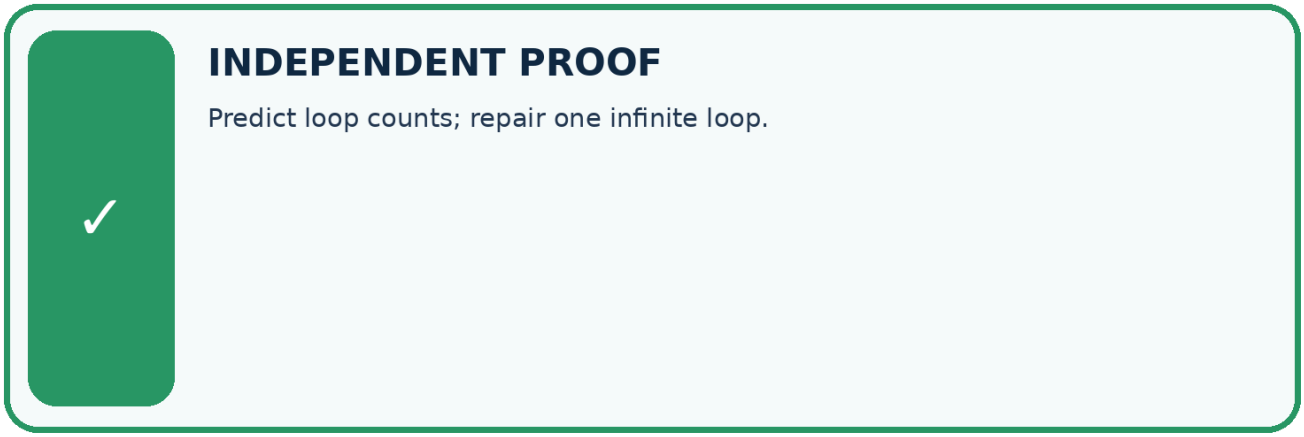
SELF-CHECK ☐ I explained the **mechanism** ☐ I used **evidence** ☐ I tested a fresh case ☐ I named a limitation ☐ I can answer 'why?'

▼ Earlier wording, alternatives and picture transcripts

INDEPENDENT MASTERY PROOF

Checklist for explaining this week's idea

Use after practice. Changed independent case is the selected independent case.



EXPLANATION CHECK Trace the pre-update condition, explain progress and boundary crossing, and inspect nonterminating variants using finite prefixes.

Write or draw your reasoning and evidence.

▼ **Extra practice: A loop must progress toward stopping**

REFERENCE · SECTION 13

Extra practice: A loop must progress toward stopping

> Try the question before opening its worked explanation.

Read the supplied case and question

Start n=0; while n<3, add 1 and record n.

List records and stopping condition.

Your explanation

- Which detail supports the answer, and why does it matter?

Write or draw your reasoning and evidence.

▼ Check after trying

1,2,3. The next comparison is false. The **update** makes progress toward termination.

Evidence or stage	Value or result
Start n	0
First update	1
Second update	2
Third update	3
Next n < 3 check	False, stop

▼ Earlier wording, alternatives and picture transcripts

Week 5 Worked case

Use the supplied details to practise the weekly concept before attempting the independent question. This case extends the illustrated lesson.

The situation

The question

Worked explanation

Explain it in your own words

Trace the supplied case visually

A loop must progress toward stopping

Start n=0; while n<3, add 1 and record n.

List records and stopping condition.

1,2,3. The next comparison is false. The **update** makes progress toward termination.

▼ Reference: original lesson checklist

REFERENCE · SECTION 15

Reference: original lesson checklist

> Use an additional checklist when it helps inspect your explanation.

Targets and evidence

By the end, I can	How I will show it
Trace variable values before and after each loop iteration	Explain the deciding step, show a trace or test, and state one limit.
Use the excluded stop value in range correctly	Explain the deciding step, show a trace or test, and state one limit.
Explain initialization and test zero one and several iterations	Explain the deciding step, show a trace or test, and state one limit.

Keep your route and first prediction

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: A fictional counter starts at 12 and decreases while above a boundary.

▼ Extra practice: pictures, repair and feedback

REFERENCE · SECTION 17

Extra practice: pictures, repair and feedback

> Attempt each question before opening its explanation. Use one browser or paper copy.

First supported question

- A fictional counter starts at 12 and decreases while above a boundary.
- Which labelled step matches this detail? 0>3 is false; report 0 after three **update**s.
- 1. Next check
- 2. Final **update**
- 3. Stop
- 4. Initial check
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: Stop 0>3 is false; report 0 after three **update**s. The condition does not promise the counter ends exactly at 3. Subtractions can cross the boundary.

The count strictly decreases, so the below-or-equal stopping region is eventually reached from this start. A condition requiring exact equality can be unsafe when repeated **update**s skip that value.

Changed-case question

- Changed-case practice: Replace count>3 with count!=3. Which conclusion is supported by the supplied facts?
- 1. We can decide the result without examining the changed **input** or the procedure.
- 2. From 12, the **loop** never reaches 3 and does not terminate in this toy model.
- 3. This one example guarantees correct results for every future **input**.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check the changed case**

After your attempt, compare with the supported explanation: From 12, the **loop** never reaches 3 and does not terminate in this toy model. 12,8,4,0,−4,... are multiples of 4; none equals 3.

Repair and save your work

Review **Trace Table**s (ai-2/4). Goal: Trace variable values before and after each **loop iteration**. Short practice: Move print inside the original **loop**. What sequence is displayed?.

Return to this week and explain what you changed.

Review Decompose a Goal (ai-2/2). Goal: Distinguish a function definition from a function call. Short practice: What happens visibly if the caller stores needed but omits print(needed)?.

Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work
</details>

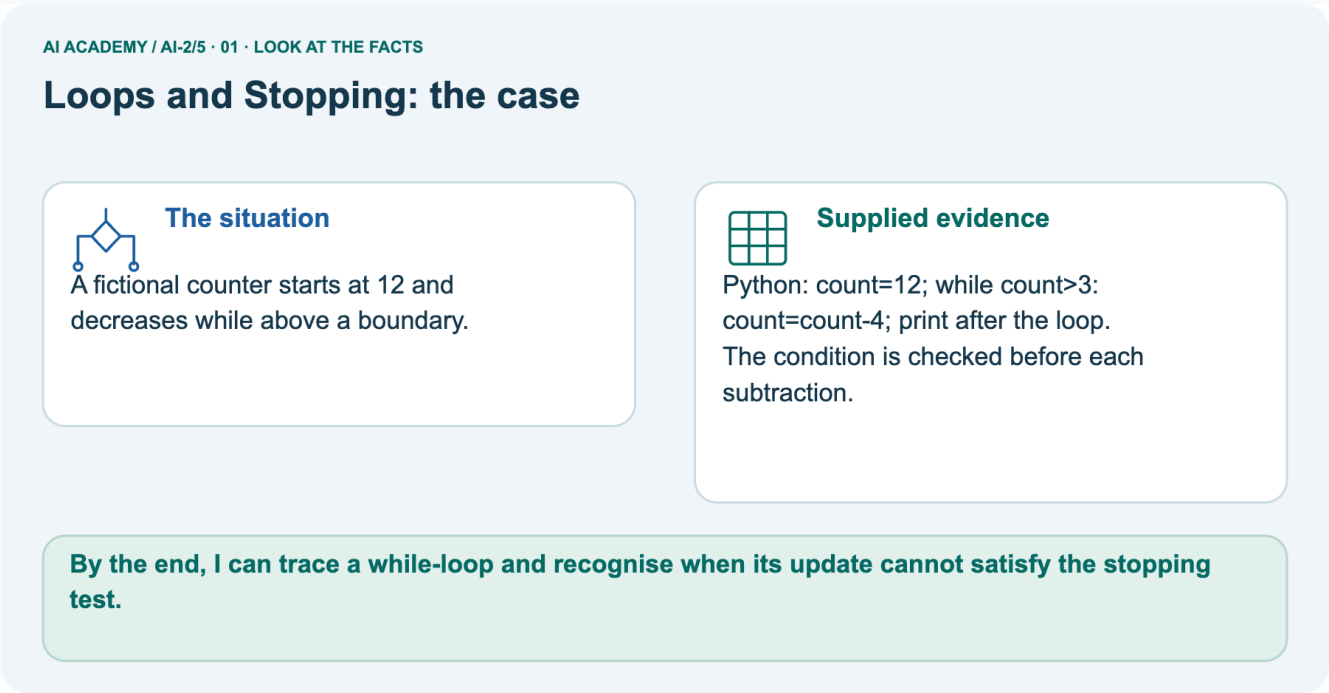
<details>
<summary>Reference: the six-picture journey</summary>
</details>

REFERENCE · SECTION 18

Reference: the six-picture journey

> Revisit the picture that serves the current question; key diagrams already sit beside the main explanation.

01. Loops and Stopping: the case

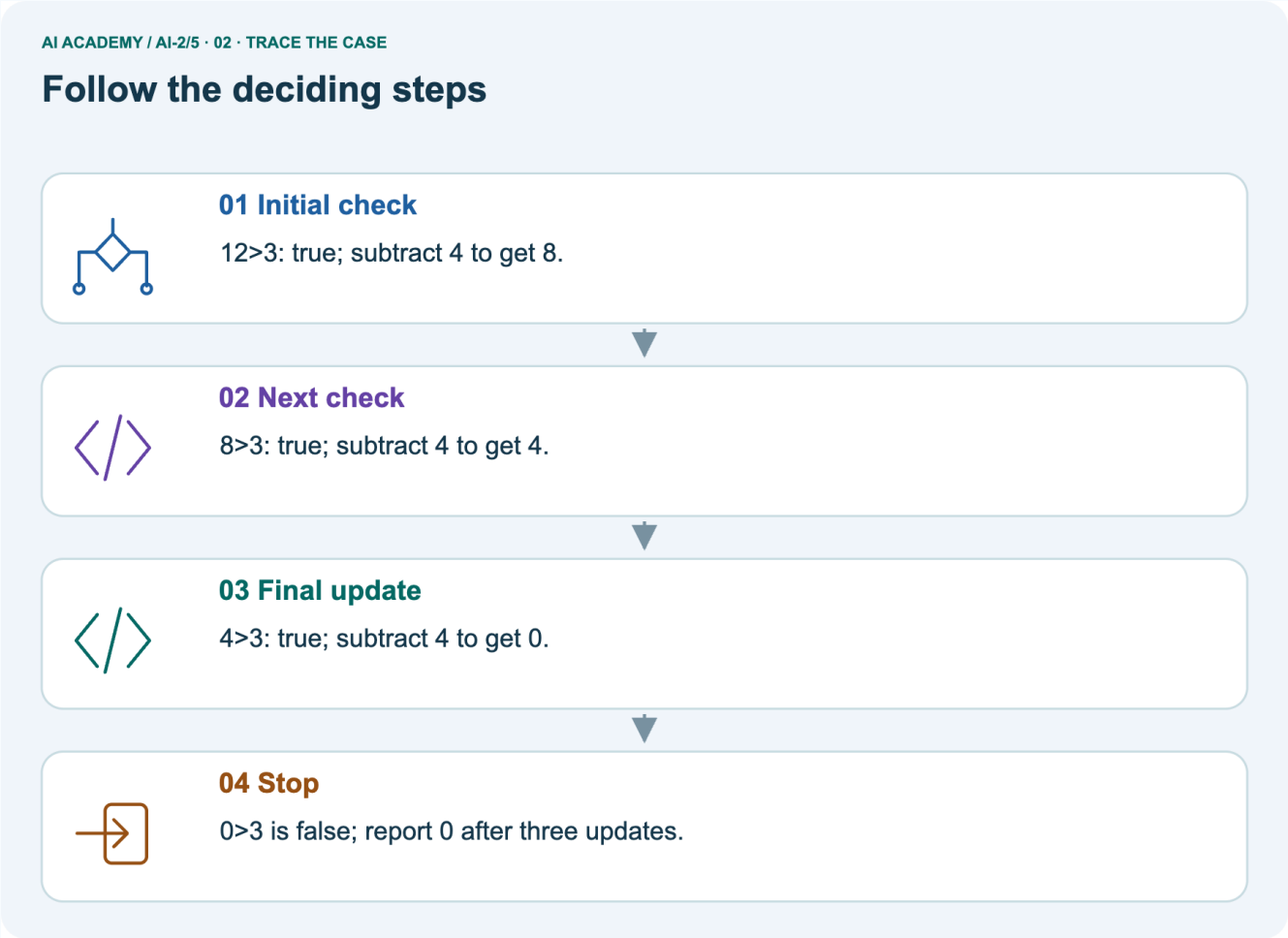


Follow the labelled picture with the matching explanation. Open the vector diagram

A fictional counter starts at 12 and decreases while above a boundary. Supplied facts: Python: count=12; while count>3: count=count-4; print after the **loop**. The condition is checked before each subtraction.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Initial check: 12>3: true; subtract 4 to get 8. → Next check: 8>3: true; subtract 4 to get 4. → Final **update**: 4>3: true; subtract 4 to get 0. → Stop: 0>3 is false; report 0 after three **update**s.

Point to each numbered stage and explain which recorded **input** it uses.

03. State after each addition

AI ACADEMY / AI-2/5 · 03 · INSPECT THE LESSON MODEL

State after each addition

turn	total before	total after
1	0	1
2	1	3
3	3	6

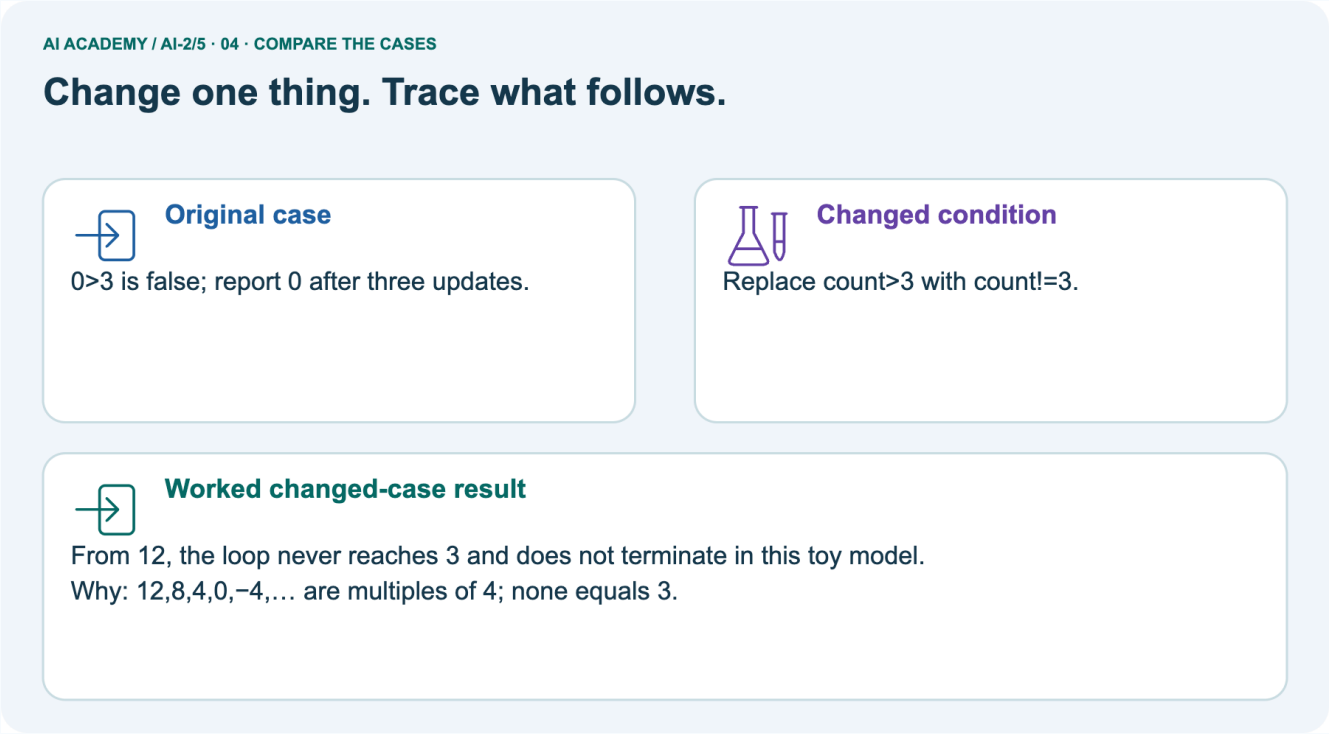
Initialize total to 0 before the loop. The unindented print runs once afterward.

Follow the labelled picture with the matching explanation. Open the vector diagram

State after each addition. 1 | 0 | 1. 2 | 1 | 3. 3 | 3 | 6

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



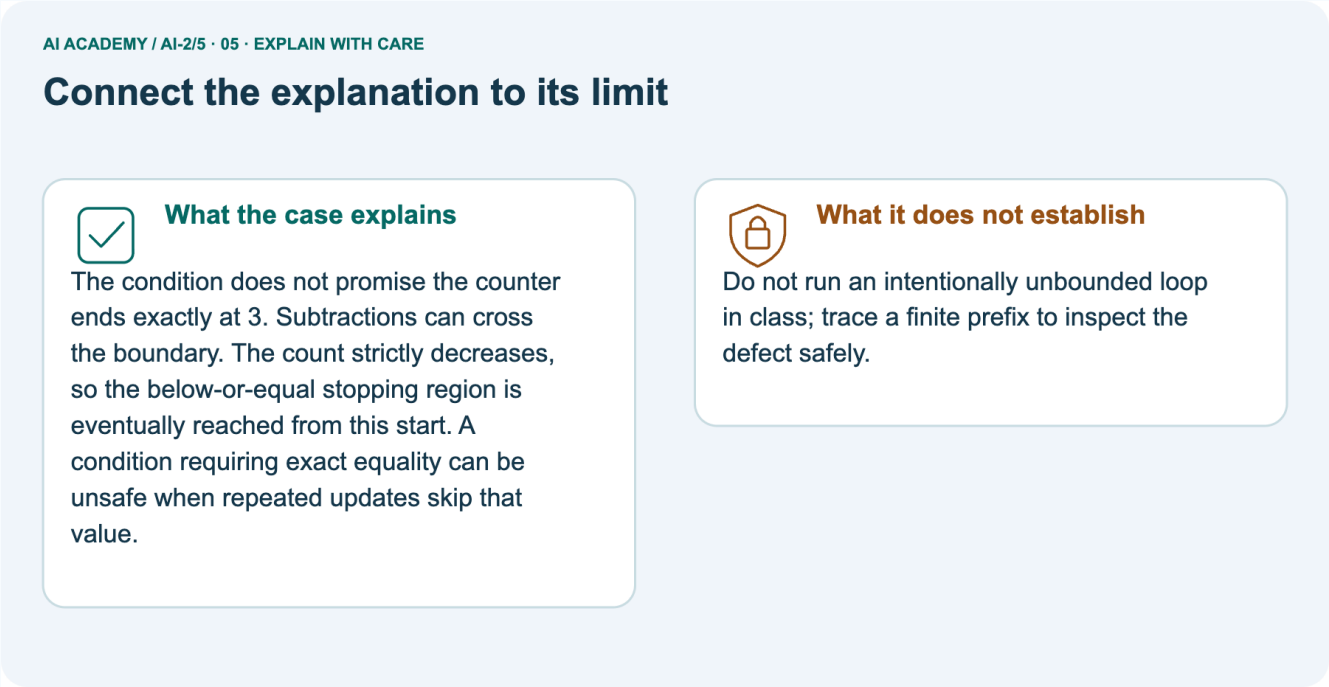
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: 0>3 is false; report 0 after three **update**s. Changed condition: Replace count>3 with count!=3. Expected result: From 12, the **loop** never reaches 3 and does not terminate in this toy model.

Reason: 12,8,4,0,−4,… are multiples of 4; none equals 3.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



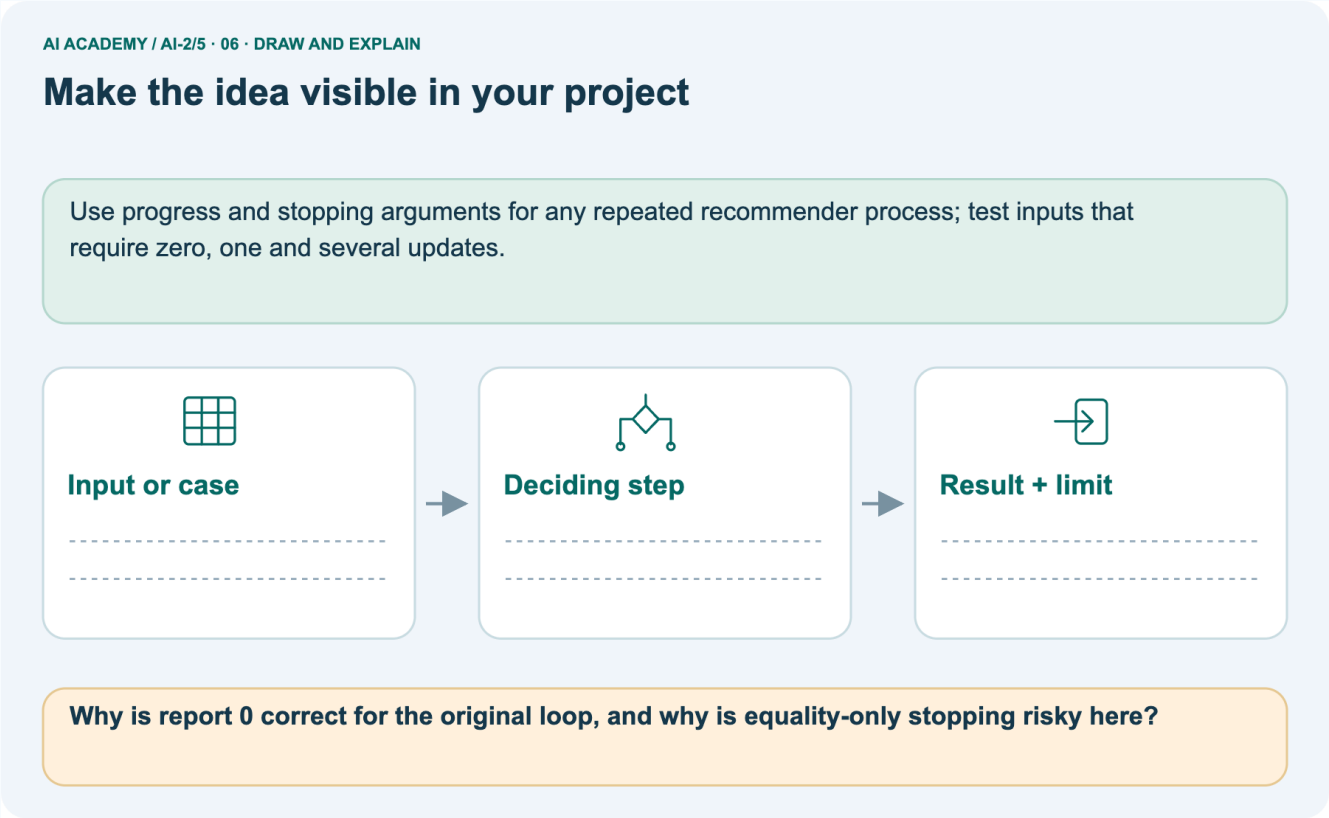
Follow the labelled picture with the matching explanation. Open the vector diagram

The condition does not promise the counter ends exactly at 3. Subtractions can cross the boundary. The count strictly decreases, so the below-or-equal stopping region is eventually reached from this start.

A condition requiring exact equality can be unsafe when repeated **update**s skip that value. Limit: Do not run an intentionally unbounded **loop** in class; trace a finite prefix to inspect the defect safely.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

Use progress and stopping arguments for any repeated recommender **process**; test **input**s that require zero, one and several **update**s. Draw your **input**, deciding step, result and limit.

Supported question: Why is report 0 correct for the original **loop**, and why is equality-only stopping risky here?

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Loops and Stopping: the case

02. Follow the deciding steps

03. State after each addition

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.